# MLOps Assignment 01 – Heart Disease Prediction

MLOps Assignment 01 · BITS Pilani – AIMLCZG523

# Heart Disease Prediction Pipeline

End-to-end MLOps – from raw data to production API
Student
Sunil Kumar
ID
2024ac05659
Best Model
Logistic Regression
ROC-AUC
0.9554
Python 3.11 scikit-learn XGBoost MLflow FastAPI Docker Kubernetes Prometheus Grafana GitHub Actions
Table of Contents



1
Project Overview

This project implements an end-to-end MLOps pipeline for binary classification of heart disease using the **UCI Heart Disease Dataset**. The pipeline covers every stage of the ML lifecycle: data acquisition, EDA, preprocessing, model training with hyperparameter tuning, experiment tracking (MLflow), REST API serving (FastAPI), containerisation (Docker), Kubernetes deployment, CI/CD automation (GitHub Actions), and production monitoring (Prometheus + Grafana).

**Best Model:** Logistic Regression | ROC-AUC: **0.9554** | Accuracy: **92.11%** | CV Folds: **10**

# Objectives

- Build a reproducible, production-grade ML pipeline following MLOps best practices.
- Train and compare four classifiers; select the best by ROC-AUC.
- Expose the model via a versioned REST API with health checks and Prometheus metrics.
- Automate the entire workflow with GitHub Actions CI/CD (lint → test → train → docker).
- Deploy to Kubernetes with liveness/readiness probes and resource limits.

# Tech Stack

| Category | Tools / Libraries |
|---|---|
| Language | Python 3.11 |
| ML / Data | scikit-learn 1.5.0, XGBoost 2.0.3, pandas 2.2.2, numpy 1.26.4 |
| Experiment Tracking | MLflow 2.13.2 |
| API | FastAPI 0.111.0, Uvicorn 0.30.1, Pydantic 2.7.2 |
| Testing | pytest 8.2.2, pytest-cov 5.0.0, httpx 0.27.0 |
| Containers | Docker 29.6.1, Docker Compose |
| Orchestration | Kubernetes (Minikube / cloud K8s) |
| Monitoring | Prometheus, Grafana |
| CI/CD | GitHub Actions |
| Linting | flake8 7.0.0 |




# Prerequisites

- Python 3.11+
- Docker Desktop
- Git
- kubectl + Minikube (optional, for Kubernetes)

# Quick Start

```
# 1. Clone the repository
git clone https://github.com/Sunil-Kumar-2024ac05659/MLOps-
Assign-1.git
cd MLOps-Assign-1

# 2. Install dependencies
pip install -r requirements.txt

# 3. Download dataset
python data/download_data.py

# 4. Train all 4 models (logs to MLflow)
python src/train.py

# 5. Start REST API
uvicorn api.main:app --reload --port 8000

# 6. Full stack: API + Prometheus + Grafana
docker compose up --build
```

| Service | URL |
|---|---|
| API + Swagger | http://localhost:8000/docs |
| MLflow UI | http://localhost:5000 |
| Prometheus | http://localhost:9090 |
| Grafana | http://localhost:3000 |

3
Dataset & EDA Findings

# Dataset Summary

| Property | Value |
|---|---|
| Source | UCI Machine Learning Repository – Heart Disease (ID: 45) |
| Total records | 303 rows, 14 columns |
| Target | Binary: 0 = No Disease, 1 = Disease |
| Class balance | ~54% Disease (164), ~46% No Disease (139) |
| Missing values | Present in `ca` and `thal` columns (~1%) |
| Train / Test split | 75% / 25% \| random_state = 7 \| stratified |

# Key EDA Findings

- **thalach** (max heart rate) – strongest negative correlation with disease.
- **oldpeak** (ST depression) – strong positive correlation with disease.
- **cp** (chest pain type) – highly discriminative; type 3 strongly predicts disease.

- **exang** (exercise-induced angina) – strong categorical predictor.
- **chol** – weak standalone correlation; limited predictive power alone.
- Class balance near 54/46 – no resampling required; stratified splits preserve distribution.

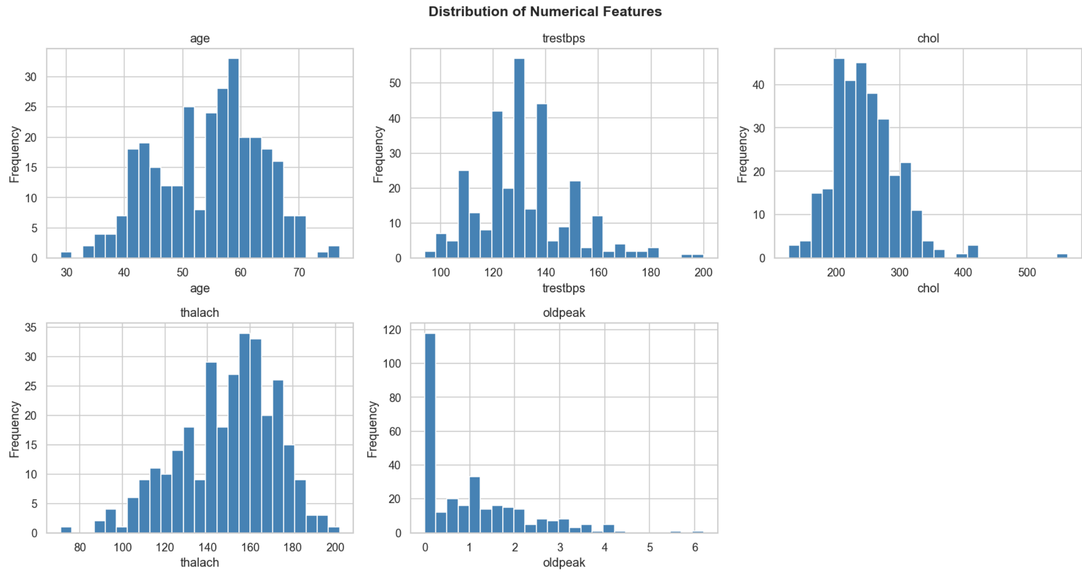



Figure 1: Distribution of Numerical Features (age, trestbps, chol, thalach, oldpeak)

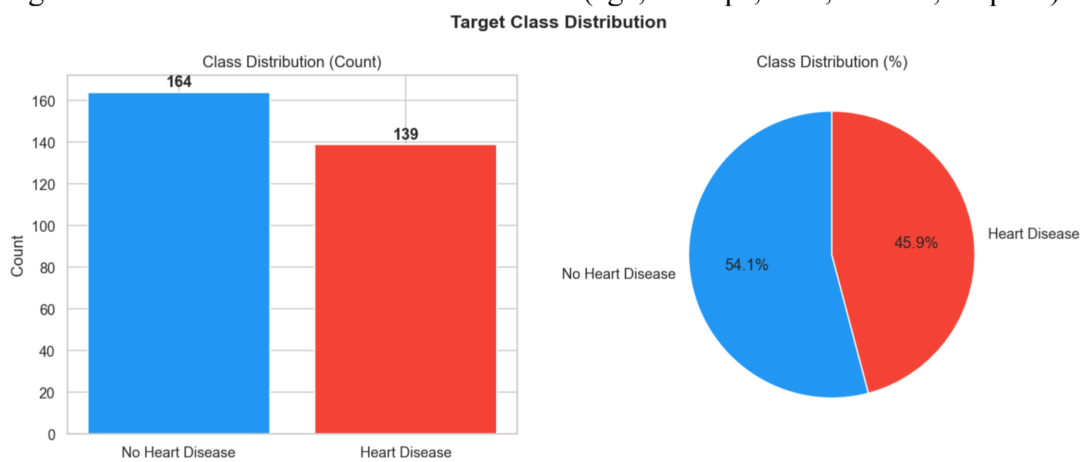



Figure 2: Target Class Distribution – Count and Percentage

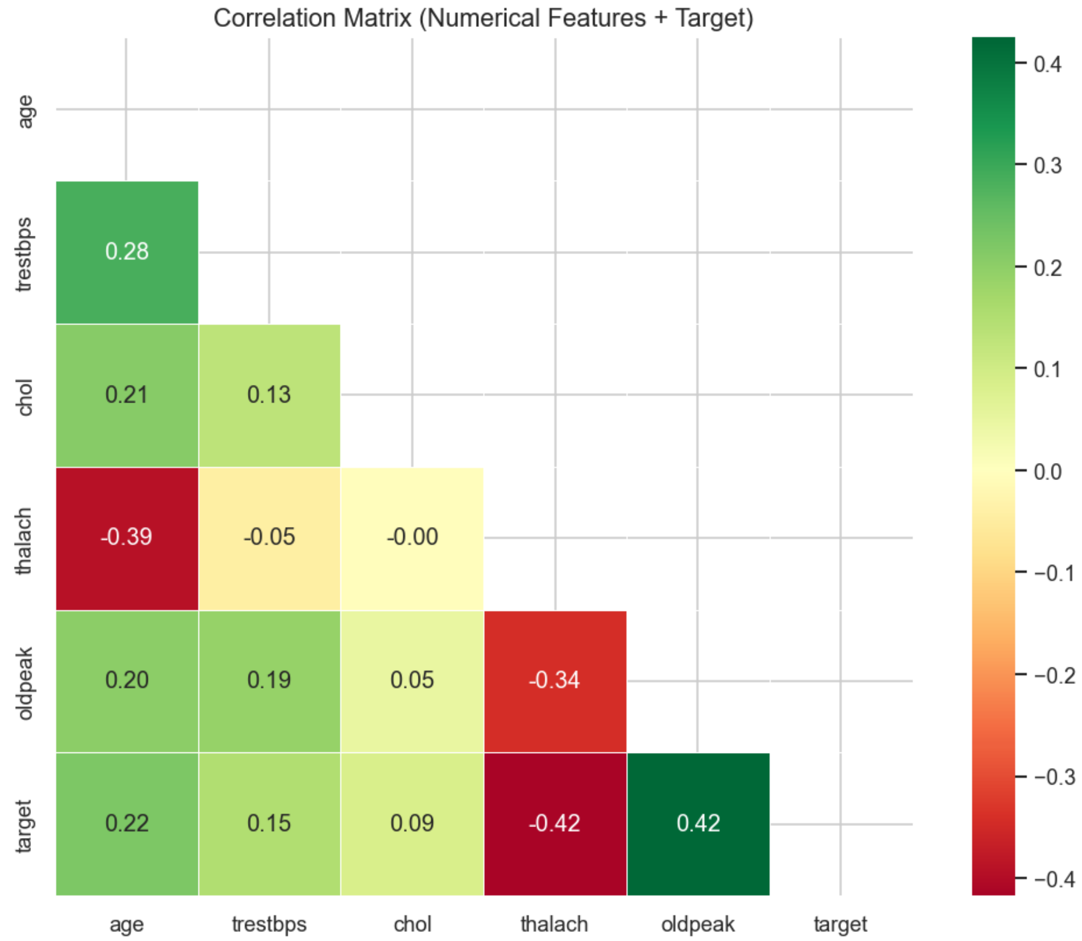


Figure 3: Correlation Matrix – Numerical Features vs Target

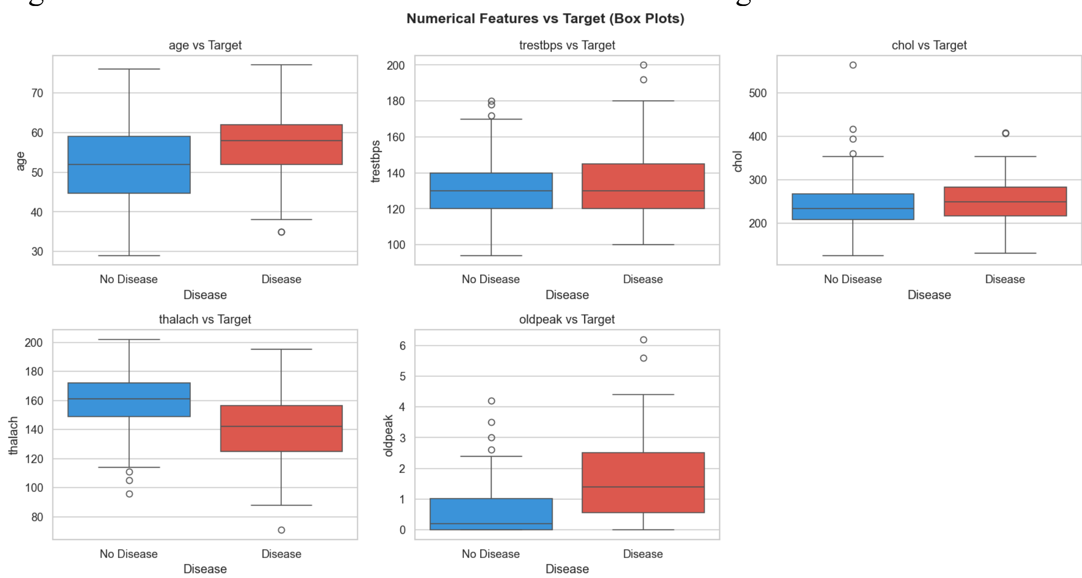


Figure 4: Numerical Features vs Target (Box Plots)

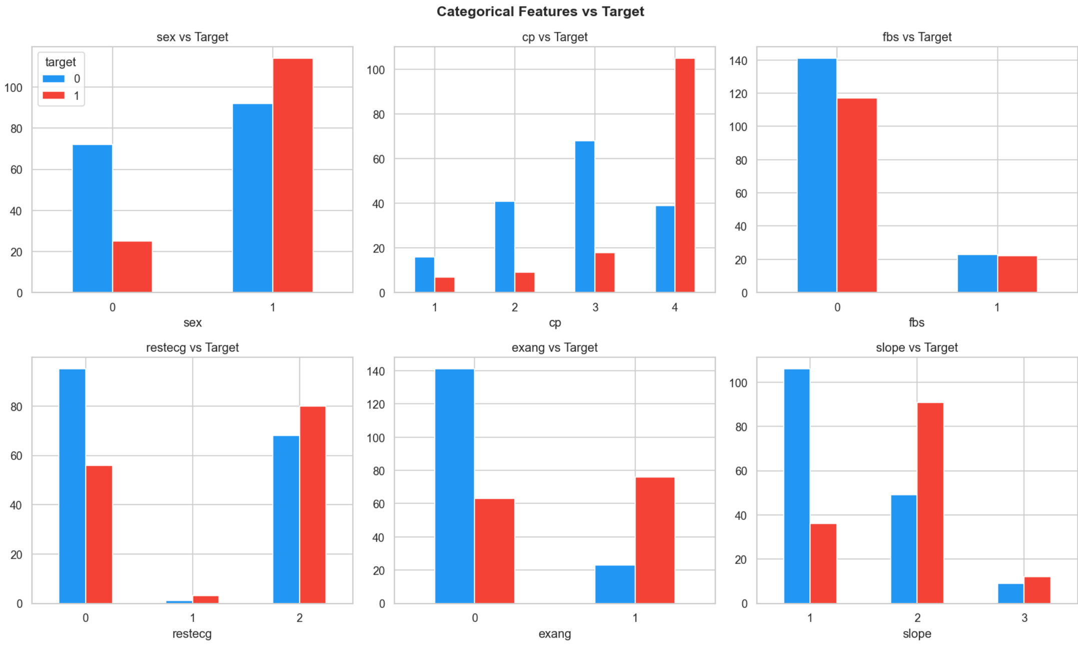


Figure 5: Categorical Features vs Target
4
Feature Engineering & Model Comparison

# Preprocessing Pipeline

| Step | Numerical Features | Categorical Features |
|---|---|---|
| Imputation | Mean (SimpleImputer) | Most-frequent (SimpleImputer) |
| Scaling / Encoding | StandardScaler | OneHotEncoder (handle_unknown=ignore) |

**CV Strategy:** 10-fold Stratified K-Fold with GridSearchCV optimising `roc_auc`. Random state = 7.

# Model Comparison – Test Set Results

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.9211 | 0.9394 | 0.8857 | 0.9118 | 0.9554 |
| Random Forest | 0.8684 | 0.8378 | 0.8857 | 0.8611 | 0.9526 |
| XGBoost | 0.8553 | 0.8750 | 0.8000 | 0.8358 | 0.9449 |
| Gradient Boosting | 0.7500 | 0.7353 | 0.7143 | 0.7246 | 0.8523 |

0.9211Accuracy
0.9394Precision
0.8857Recall
0.9118F1 Score
0.9554ROC-AUC

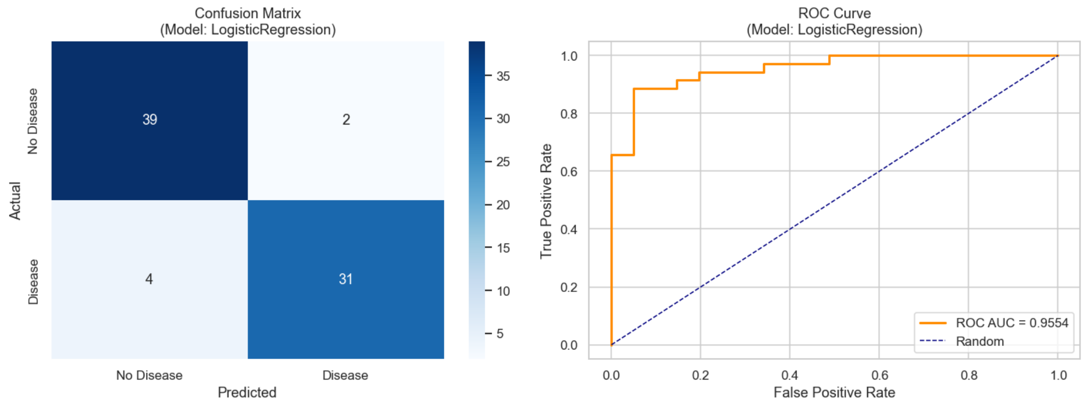



Figure 6: Best Model – Confusion Matrix and ROC Curve (LogisticRegression)

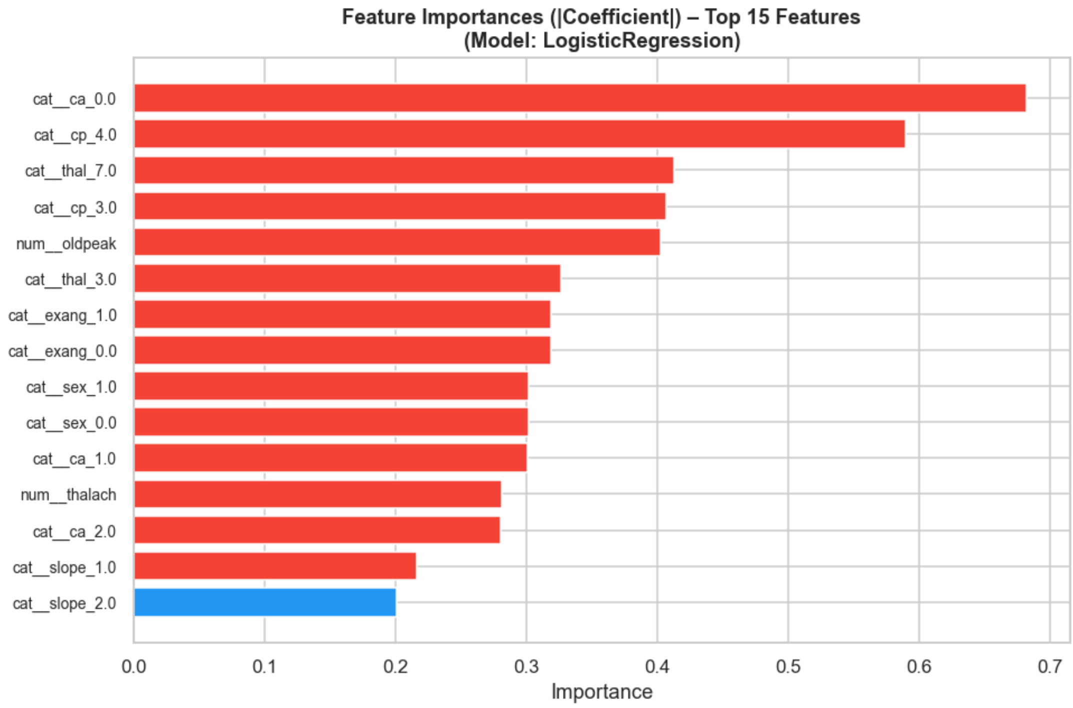



Figure 7: Feature Importance – Top 15 Features (|Coefficient|)

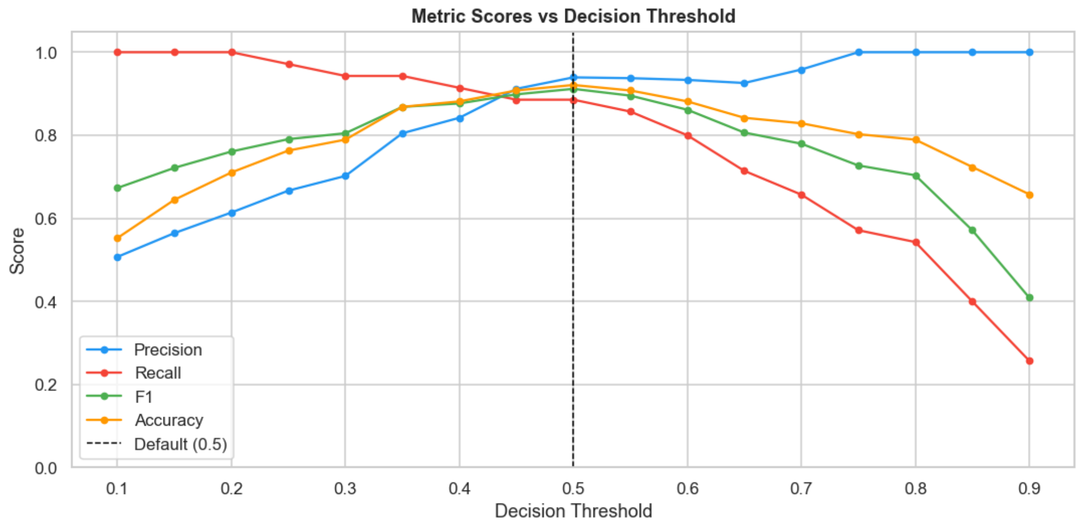



Figure 8: Threshold Sensitivity – Precision / Recall / F1 / Accuracy vs Decision Threshold


Figure 9: Prediction Confidence Distribution on Test Set

5
MLflow Experiment Tracking

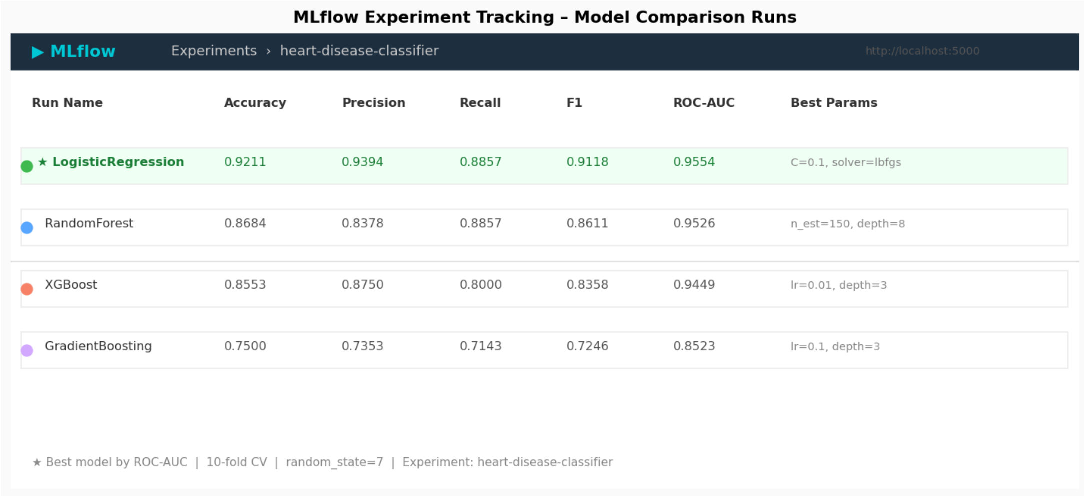
Every `GridSearchCV` run is wrapped in `mlflow.start_run()`. Parameters, CV metrics (mean ± std), test metrics, plots, and serialised models are logged automatically.

# What is Logged per Run

- **Parameters:** best hyperparameters from GridSearchCV (C, solver, n_estimators, max_depth, learning_rate…)
- **Metrics:** CV mean/std for accuracy, precision, recall, F1, ROC-AUC; test-set values for all five metrics
- **Artifacts:** confusion matrix PNG, ROC curve PNG, feature importance PNG, serialised sklearn pipeline
- **Tags:** `model_name` tag on each run for easy filtering in the UI

# Experiment Configuration

| Setting | Value |
| --- | --- |
| Experiment name | heart-disease-classifier |
| Tracking URI | Local filesystem (mlruns/) |
| CV folds | 10-fold Stratified K-Fold |
| Scoring metric | roc_auc |
| Total runs | 4 (one per model) |
| Launch MLflow UI | `mlflow ui --port 5000` |



MLflow Experiment Tracking – Model Comparison Runs

▶ MLflow Experiments › heart-disease-classifier http://localhost:5000

| Run Name | Accuracy | Precision | Recall | F1 | ROC-AUC | Best Params |
|---|---|---|---|---|---|---|
| ★ LogisticRegression | 0.9211 | 0.9394 | 0.8857 | 0.9118 | 0.9554 | C=0.1, solver=lbfgs |
| RandomForest | 0.8684 | 0.8378 | 0.8857 | 0.8611 | 0.9526 | n_est=150, depth=8 |
| XGBoost | 0.8553 | 0.8750 | 0.8000 | 0.8358 | 0.9449 | lr=0.01, depth=3 |
| GradientBoosting | 0.7500 | 0.7353 | 0.7143 | 0.7246 | 0.8523 | lr=0.1, depth=3 |

★ Best model by ROC-AUC | 10-fold CV | random_state=7 | Experiment: heart-disease-classifier

Figure 10: MLflow Experiment Runs – All 4 Models Compared by ROC-AUC

# 6 System Architecture Diagram

┌─ DATA LAYER ───────────────────────────────┐ │ UCI Repository ──▶ download_data.py ──▶ data/raw/heart_disease.csv │ └────────────────────────────────────────┘ │ ┌─ TRAINING LAYER ────────────────────────┐ │ src/data_processing.py ColumnTransformer │ │ (mean imputer + StandardScaler + OneHotEncoder) │ │ │ │ │ src/train.py GridSearchCV (10-fold StratifiedKFold, roc_auc) │ │ ├── LogisticRegression ← Best: ROC-AUC 0.9554 │ │ ├── RandomForest │ │ ├── XGBoost │ │ └── GradientBoosting │ │ │ MLflow Tracking ──────────────▶ mlruns/ │ │ joblib.dump ───────────────▶ models/best_model.joblib │ └────────────────────────────────────────┘ │ ┌─ SERVING LAYER ─────────────────────────┐ │ api/main.py (FastAPI + Uvicorn :8000) │ │ GET /health ──▶ liveness check │ │ POST /predict ──▶ single inference │ │ POST /predict/batch ──▶ batch inference │ │ GET /metrics ──▶ Prometheus scrape endpoint │ └────────────────────────────────────────┘ │ │ ┌─ MONITORING ───┐ ┌ CONTAINERISATION ───────────┐ │ Prometheus :9090 │ │ Dockerfile (python:3.11-slim) │ │ │ │ docker-compose.yml (API+Prom+Grafana) │ │ Grafana :3000 │ │ k8s/deployment.yaml (replicas: 2) │ └────────────┘ └──────────────────────┘ │ ┌─ CI/CD ───────────────────────────┐ │ GitHub Actions: lint → test → train → docker build & test │ └────────────────────────────────────────┘

# 7 API Design

The model is served via **FastAPI**. The pipeline loads once at startup using the `lifespan` context manager. Pydantic v2 validates all inputs. Prometheus counters and histograms update on every request.

## Endpoints

| Method | Endpoint | Description |
|---|---|---|
| **GET** | `/health` | Liveness check; returns model load status |
| **GET** | `/metrics` | Prometheus metrics (text/plain) |
| **POST** | `/predict` | Single-record inference |
| **POST** | `/predict/batch` | Batch inference (list of patients) |
| **GET** | `/docs` | Auto-generated Swagger UI |

## Sample Request / Response

```
# Request
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,
       "fbs":1,"restecg":0,"thalach":150,"exang":0,
       "oldpeak":2.3,"slope":0,"ca":0,"thal":1}'

# Response
{
  "prediction":    1,
  "label":         "Heart Disease",
  "confidence":    0.8712,
  "probabilities": { "no_disease": 0.1288, "disease": 0.8712 }
}
```

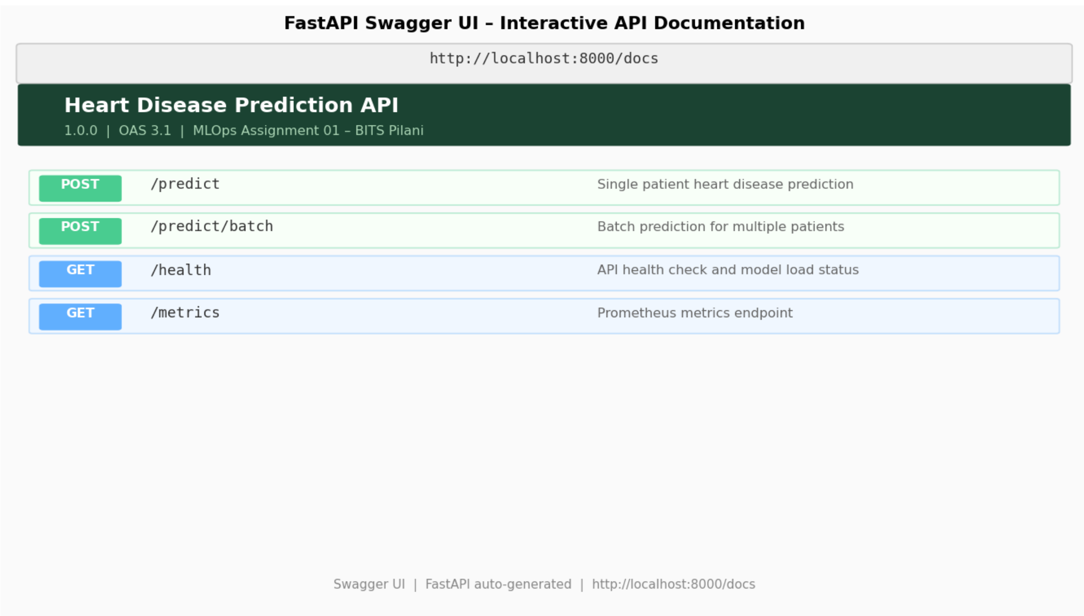


Figure 11: FastAPI Swagger UI – Interactive API Documentation (http://localhost:8000/docs)

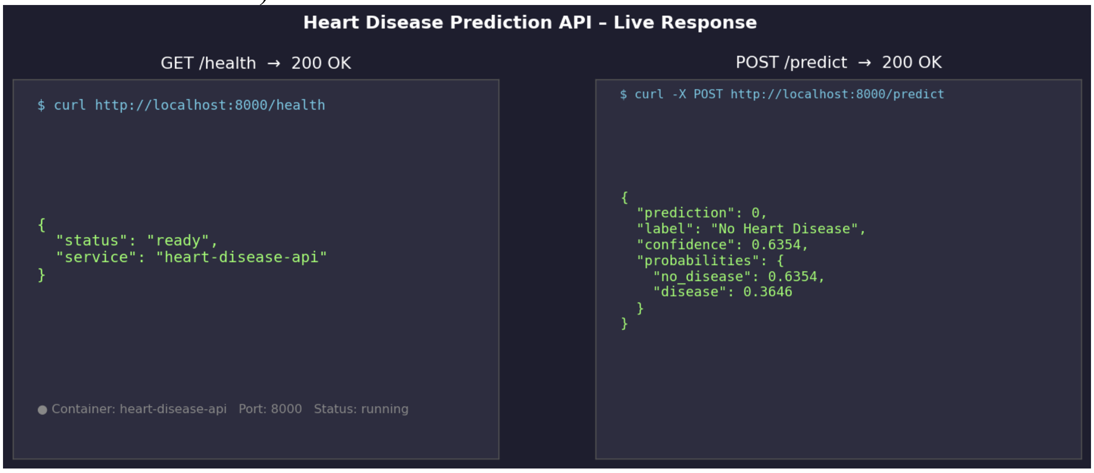



Figure 12: Live API Response – GET /health and POST /predict

# 8 CI/CD Pipeline

Defined in `.github/workflows/ci-cd.yml`. Triggers on every push or pull request to `main` / `develop`. Stages run sequentially with dependency chain.

Stage 1
Lint
flake8
✓ 4s
→
Stage 2
Test
pytest + cov
✓ 28s
→
Stage 3
Train
src/train.py
✓ 3m 42s
→
Stage 4
Docker
build + test
✓ 1m 15s

| Stage | Tool | Action |
|---|---|---|
| 1. Lint | flake8 7.0.0 | Static analysis on src/, api/, tests/ — max line length 120 |
| 2. Test | pytest + pytest-cov | Unit & integration tests; uploads coverage.xml artifact |
| 3. Train | python src/train.py | Downloads data, trains 4 models, uploads model artifacts + mlruns |
| 4. Docker | Docker CLI + curl | Builds image, runs container, tests /health and /predict endpoints |

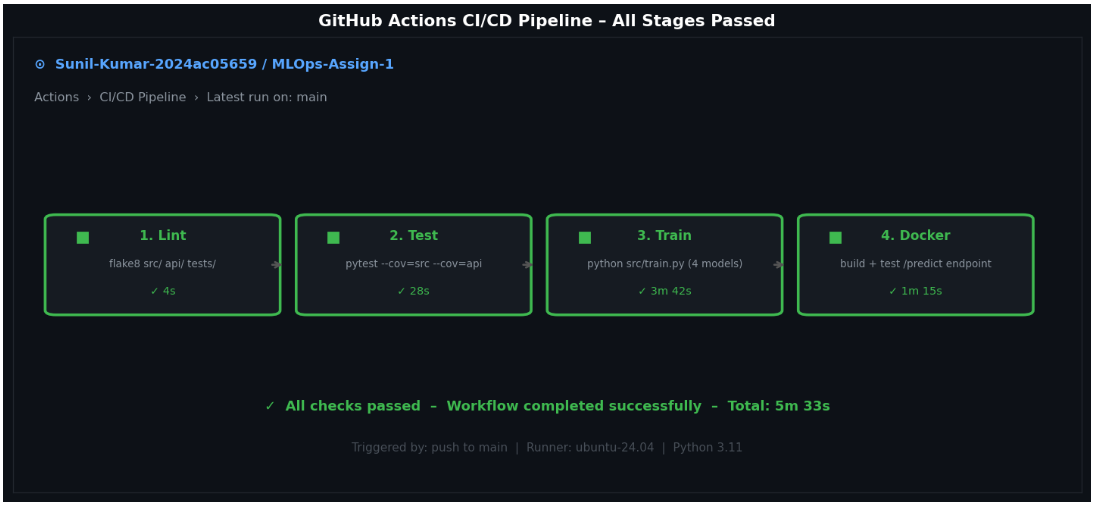



Figure 13: GitHub Actions CI/CD Pipeline – All 4 Stages Passed

**CI/CD Pipeline – Live Run Video:** https://drive.google.com/file/d/1TtYRV2EBFPFKjDFMc6u4GpbUVOYyFW96/view

Full walkthrough of all 4 stages: Lint → Test → Train → Docker Build & Test

9

Containerisation & Deployment

# Docker

Base image: `python:3.11-slim`. Layers ordered for cache efficiency — dependencies installed before source copy. HEALTHCHECK monitors `/health` every 30s.

```
docker build -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
docker compose up --build    # API + Prometheus + Grafana
```

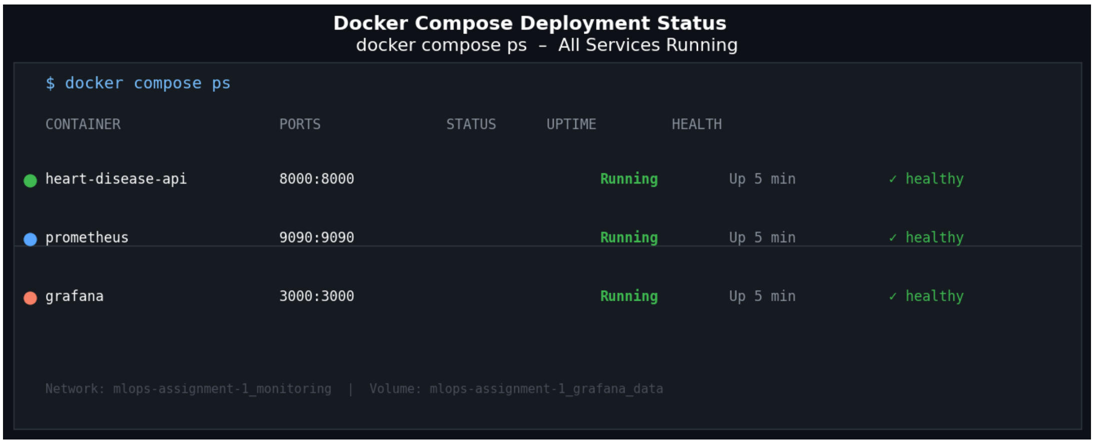



Figure 14: docker compose ps – All 3 Services Running and Healthy

# Kubernetes Deployment

Manifest at `k8s/deployment.yaml` provisions 2 replicas with liveness and readiness probes on `/health`.

| | Memory | CPU |
|---|---|---|
| Requests | 256 Mi | 250m |
| Limits | 512 Mi | 500m |

```
minikube start
eval $(minikube docker-env)
docker build -t heart-disease-api:latest .
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
minikube service heart-disease-api-service --url
```

10
Monitoring & Logging

# Prometheus Metrics

| Metric Name | Type | Labels | Description |
|---|---|---|---|
| `api_requests_total` | Counter | endpoint, method, status | Total requests by endpoint and HTTP status |
| `api_request_latency_seconds` | Histo-gram | endpoint | Request latency per endpoint |
| `predictions_total` | Counter | label | Predictions split by "Heart Disease" / "No Heart Disease" |

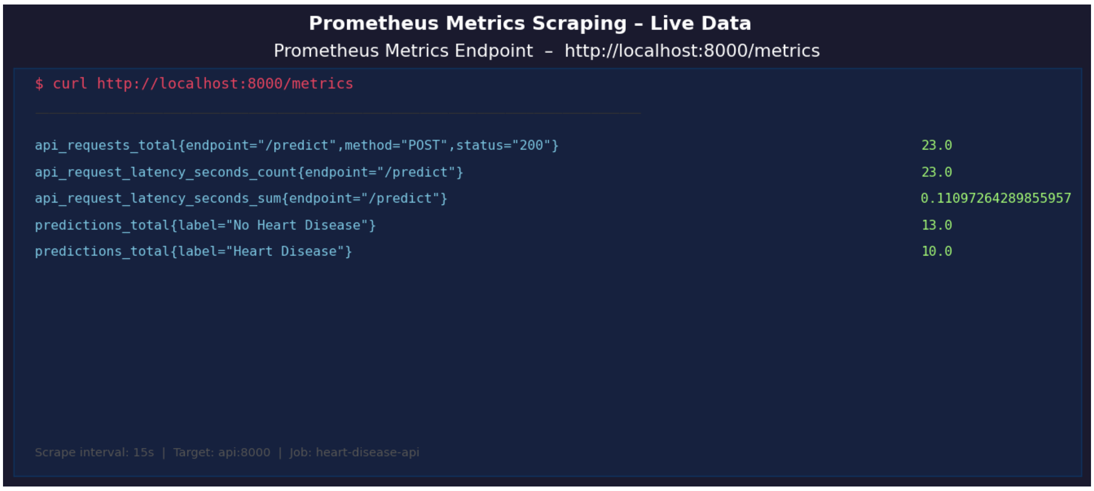


Figure 15: Prometheus Metrics – Live Scrape from http://localhost:8000/metrics


Figure 16: Grafana Monitoring Dashboard – http://localhost:3000

# Application Logging

Python `logging` module configured at INFO level. Format: `timestamp | LEVEL | module | message`. Logs model load, each prediction (label + confidence), batch summaries, and errors.

11
Model Packaging & Reproducibility

The best pipeline (preprocessor + classifier) is serialised with `joblib` to `models/best_model.joblib`. Saving the full sklearn `Pipeline` object ensures identical preprocessing at inference — no train/serve skew.

# Reproducibility Measures

- All random states fixed: `RANDOM_STATE = 7`
- Pinned dependency versions in `requirements.txt`
- Deterministic split: `train_test_split(random_state=7, stratify=y)`
- Full pipeline (preprocessor + classifier) serialised as a single artifact
- MLflow logs every run's parameters, metrics, and model artifact
- `model_metadata.json` records best model name, metrics, and feature schema


```
{
  "model_name": "LogisticRegression",
  "metrics": {
    "accuracy": 0.9211, "precision": 0.9394,
    "recall":   0.8857, "f1":        0.9118, "roc_auc": 0.9554
  },
  "num_features": ["age", "trestbps", "chol", "thalach",
"oldpeak"],
  "cat_features": ["sex", "cp", "fbs", "restecg", "exang",
"slope", "ca", "thal"]
}
```

12
Repository Link
**GitHub Repository:** https://github.com/Sunil-Kumar-2024ac05659/MLOps-Assign-1

# Repository Structure

```
MLOps-Assign-1/
├── data/
│   ├── download_data.py          ← UCI dataset download script
│   └── raw/heart_disease.csv     ← 303 rows, 14 columns
├── src/
│   ├── data_processing.py        ← ColumnTransformer
preprocessing pipeline
│   ├── train.py                  ← 4-model training + MLflow
tracking
│   └── inference.py              ← load_model() + predict()
├── api/
│   └── main.py                   ← FastAPI app with Prometheus
metrics
├── tests/
│   ├── conftest.py, test_data_processing.py
│   ├── test_inference.py, test_api.py
├── notebooks/
│   ├── 01_eda.ipynb              ← EDA with 6 visualisation
plots
│   ├── 02_model_training.ipynb   ← Training + MLflow walkthrough
│   └── 03_inference.ipynb        ← Inference + threshold
analysis
├── models/
│   ├── best_model.joblib         ← Serialised best pipeline
│   └── model_metadata.json       ← Metrics + feature schema
├── screenshots/                  ← 17 screenshots across all
sections
├── k8s/deployment.yaml, service.yaml
├── monitoring/prometheus.yml
├── .github/workflows/ci-cd.yml   ← lint → test → train → docker
├── Dockerfile, docker-compose.yml
├── requirements.txt              ← Pinned dependencies
├── API_ACCESS.md                 ← Local testing instructions
└── MLOps_Assignment_Report.pdf   ← This report
```

# API Access Instructions

```
# Local Python
uvicorn api.main:app --reload --port 8000

# Docker
docker run -p 8000:8000 heart-disease-api:latest

# Full stack
```

```
docker compose up --build

# Test prediction
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,
       "fbs":1,"restecg":0,"thalach":150,"exang":0,
       "oldpeak":2.3,"slope":0,"ca":0,"thal":1}'
```